ملخص

هذا الباب يتناول أحد أشـــهر برتوكولات الاتصـــال في عـالـم الانـترنـت وهـو WebSocket حيث يبدأ بنبذة تاريخية عن نشـــأة البرتوكول وتطوره مرورا بأهم خصائصه ومميزاته وعيوبه.

م. ربيع الشويهدي

انترنت الأشياء الفصل السابع قسم التحكم الالي

برتوكول الاتصال WebSocket

انترنت الأشياء الفصل السابع قسم التحكم الالي

1. مقدمة تاريخية

كما ذُكر في الباب السابق، يُعد بروتوكول **HTTP** البروتوكول الأكثر شيوعاً في شبكات الإنترنت، إلا أن التطور التقني وظهور تطبيقات حديثة — مثل تطبيقات إنترنت الأشياء **(IoT)** التي تتطلب مراقبة آنية **(Real-Time)** كمراقبة درجة حرارة مريض — فرضت الحاجة إلى اتصال مستمر بين العميل **(Client)** والخادم**(Server).**

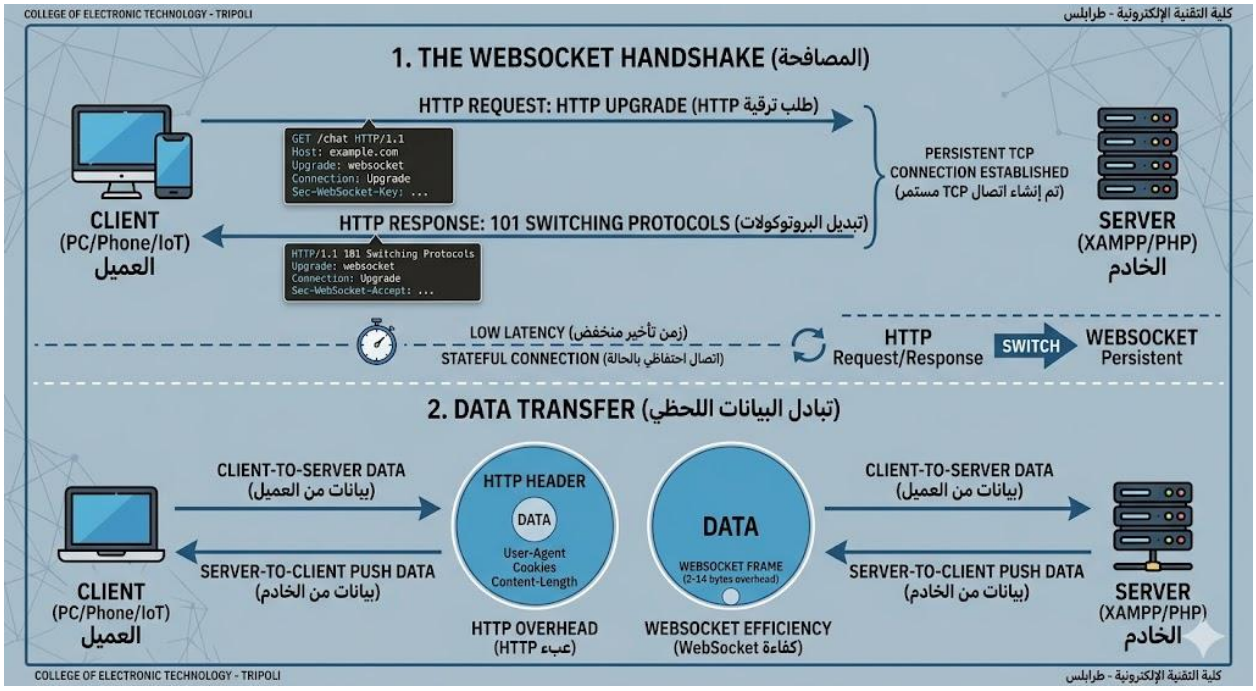
يعتمد بروتوكول **HTTP** على نموذج الطلب والاستجابة**(Request/Response)** ، وهو نموذج غير احتفاظي بالحالة**(Stateless)** ، مما يجعله غير مناسب بطبيعته للتطبيقات التي تتطلب اتصالاً مستمراً وفورياً. وقد حاول المطورون تجاوز هذه القيود باستخدام تقنيات مختلفة للحفاظ على الاتصال، إلا أنهم واجهوا عدة تحديات، من أهمها:

- **استهلاك الموارد :**يستخدم الخادم **(Server)** عدداً من قنوات الاتصال عبر بروتوكول**TCP** ، حيث تُستخدم قناة لإرسال المعلومات إلى العميل، وقد يتم إنشاء قناة جديدة لكل رسالة قادمة من العميل.
- **العبء الإضافي :(Overhead)** وجود عبء إضافي مرتفع **(High Overhead)** في بروتوكول النقل؛ حيث تحتوي كل رسالة على ترويسة**.(Headers) HTTP**
- **تعقيد البرمجيات :**الحاجة إلى تنفيذ آليات ربط **(Mapping)** في جهة العميل لربط الطلبات بالاستجابات، مما يزيد من تعقيد الشفرة البرمجية.

فكانت هذه التحديات دافعاً لتطوير بروتوكول الاتصال **WebSocket**؛ حيث تتمثل أهم ميزاته في قدرته على فتح قناة اتصال مستمرة طالما كانت هناك حاجة لذلك، مما جعله ينتشر بشكل سريع كونه بروتوكول اتصال يسمح بالإرسال والاستقبال في الوقت نفسه**(Full-Duplex)** ، علاوة على كونه بروتوكولاً خفيفاً مقارنة ببروتوكول **HTTP**. كما أن الميزة الأهم التي سهّلت سرعة انتشاره هي حرص المطورين على أن يعمل على البنية التحتية القائمة أصلاً في عالم الإنترنت، وذلك من خلال توافقه الكامل مع بروتوكول**HTTP** .

2. طريقة العمل

يبدأ اتصـال **WebSocket** بما يعرف بعملية **المصـافحة(Handshake)** ؛ حيث يرسـل العميل طلباً عادياً عبر بروتوكول **HTTP** يحتوي على **Upgrade**، يطلب فيها من الخادم تغيير بروتوكول الاتصال. إذا وافق الخادم، يتم تبديل الاتصال فوراً إلى قناة **WebSocket** مستمرة، ومن تلك اللحظة تختفي **(Headers)**الثقيلة، ويصبح تبادل البيانات سريعاً وخفيفاً جداً.

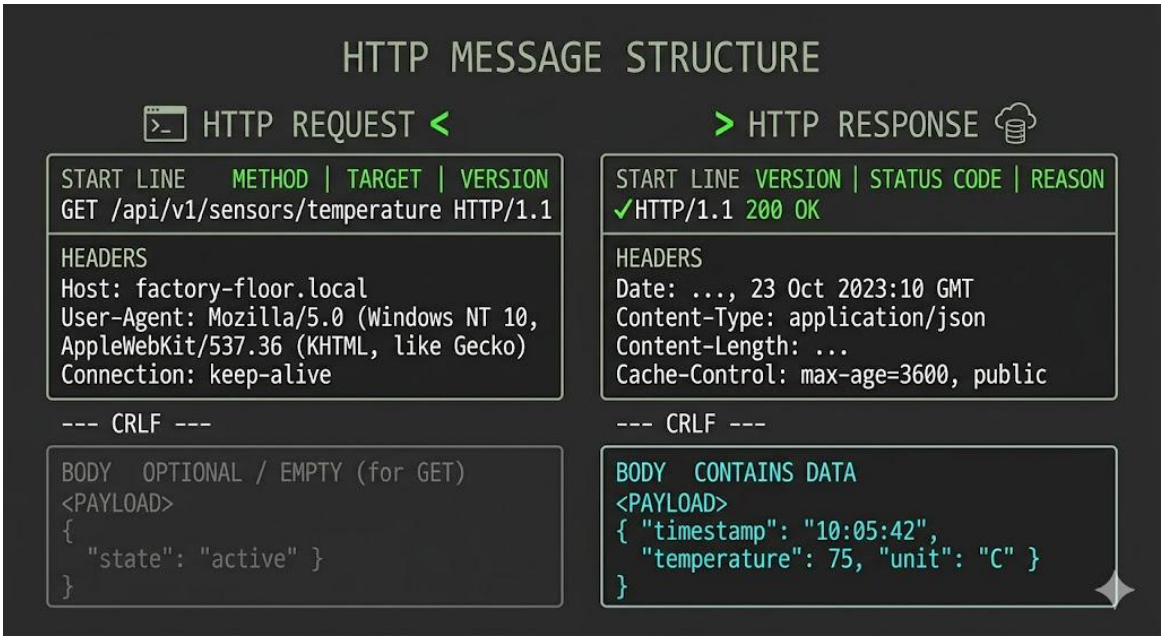


3. الخصائص البنيوية والأمنية لبروتوكولWebSocket

صُمم بروتوكول WebSocket ليكون حلاً هندسياً متكاملاً يتجاوز مجرد نقل البيانات، وتتجلى قوته في المزايا التقنية التالية:

1. **الأمان المرتكز على المصدر (Origin-based Security)** يُعزز عامل الأمان في عملية الاتصال من خلال ضمان أن الاتصال لا يتم إلا بين طرفين موثوقين، وذلك باستخدام مفهوم الأمان القائم على المصدر، مما يحمي النظام من محاولات الاتصال غير المصرح بها.
2. **تعدد الخدمات والمشاركة عبر المنفذ الواحد (Addressing & Protocol Naming)** تسمح هذه التقنية بتمرير عدة خدمات عبر منفذ واحد(**Port**) ؛ حيث صُمم **WebSocket** ليشارك المنافذ مع بروتوكولات الاتصال الأخرى، مثل المنفذ (80) مع بروتوكول **HTTP.** كما يُمكّن من تشغيل عدة مشاريع بعناوين مختلفة على نفس العنوان(**IP**) ، مثل:
   - مشروع تحكم في الإضاءة **lab1.cet.edu.ly**
   - مشروع مراقبة محرك **lab2.cet.edu.ly**
3. **آلية التأطير :(Framing Mechanism)** على عكس بروتوكول **TCP** الذي يتعامل مع البيانات كدفق مستمر (**Stream**) من البايتات دون حدود واضحة بين الرسائل — مما قد يضطر المطور أحياناً للقيام بعملية فك وتصنيف يدوي للبيانات المستلمة — يقوم بروتوكول **WebSocket** بإضافة طبقة تأطير (**Framing**) ذكية. هذه الطبقة تقوم بتغليف البيانات في وحدات مستقلة تسمى أُطراً (**Frames**)، حيث يحتوي كل إطار على ترويسة (**Header**) بسيطة تتضمن:
   - نوع البيانات (**Opcode**): يحدد ما إذا كان الإطار نصياً، ثنائياً (**Binary**)، أو إطار تحكم (مثل إغلاق الاتصال).
   - حجم البيانات (**Payload Length**): يخبر المستلم بالضبط أين تنتهي الرسالة الحالية.
   - قناع الحماية (**Masking Key**): (في الرسائل الصادرة من العميل) لضمان أمن البيانات ومنع التداخل."
4. **إنهاء الاتصال عبر (Closing Handshake)** : يتضمن البروتوكول عملية إغلاق منظمة للاتصال(**In-band**)، صُممت خصيصاً لتعمل بكفاءة حتى في وجود الخوادم الوسيطة (**Proxies**) والوسائط التقنية الأخرى، وذلك لضمان إنهاء الاتصال بشكل سليم ومنسق من كلا الطرفين.

- ملاحظة: بروتوكول **WebSocket** هو بروتوكول مبني على بروتوكول **TCP** (**TCP-based protocol**) مع هذا فهو بروتوكول مستقل تماما، علاقته مع بروتوكول **HTTP** تنحصر في عملية المصافحة (**Handshaking**)



4. نظام البرمجة القائم على الأحداث (Event-Driven Programming)

على خلاف بروتوكول **HTTP** الذي يعتمد على نموذج "الطلب/الاستجابة" (**Request/Response**)، حيث يستخدم مجموعة من الطرق (**Methods**) مثل **GET** و**POST** و**PUT**، والتي تُعد جزءًا أساسيًا من ترويسة الطلب (**Request Header**)، فإن بروتوكول **WebSocket Protocol** لا يحتوي على هذا المفهوم بعد إنشاء الاتصال، بل يوفر قناة اتصال مستمرة (**Persistent Connection**) لتبادل الرسائل (**Messages**) بين الطرفين.

ويركّز البروتوكول على نقل البيانات وتأطيرها (**Framing**) دون فرض قيود على نوع البيانات، بينما يتم تحديد طول الرسالة ضمن بنية الإطار (**Frame**).

ونظراً للتعقيد البرمجي المرتبط بالتعامل المباشر مع بروتوكول **WebSocket** (على مستوى الـ **Framing** والـ **Handshaking**)، فقد تم تطوير واجهات برمجية (**APIs**) تعتمد على النموذج القائم على الأحداث (**Event-Driven Model**)، مثل (**WebSocket API**). تقوم هذه الواجهات بتجريد (**Abstracting**) حالات الاتصال المعقدة — مثل فتح القناة، استقبال الأطر البرمجية، أو إنهاء الجلسة — وتحويلها إلى أحداث برمجية (**Events**)

يســـهل التعامل معها برمجياً. في بيئة المتصـــفحات، لا يتم التعامل مع بروتوكول **WebSocket** بشـــكل مباشر (**Low-level**)، بل يتم ذلك حصرياً من خلال هذه الواجهات التي تُعد الوسيلة المعيارية والتلقائية للتفاعل معه.

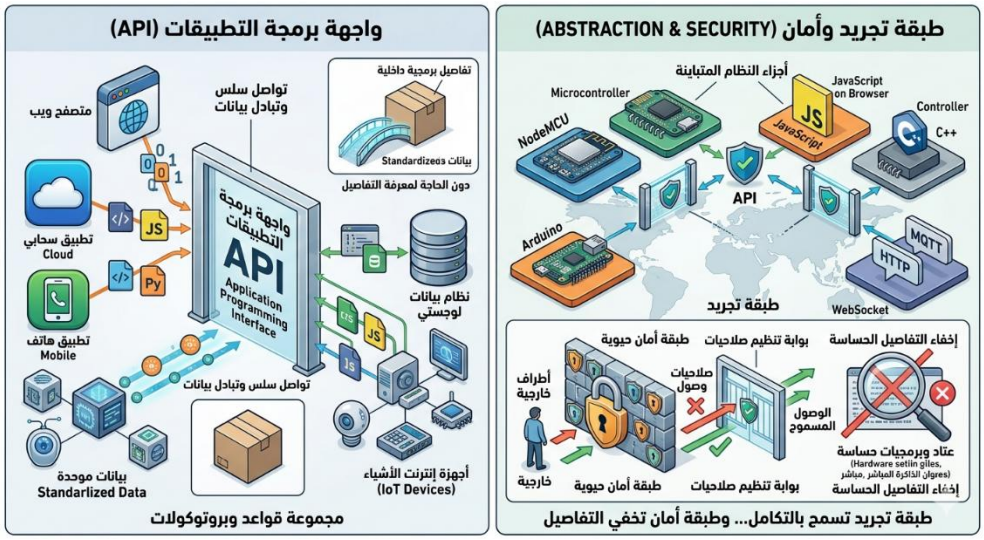
أما في الأنظمة المدمجة (مثل **ESP32**)، فبالرغم من إمكانية التعامل مع البروتوكول برمجياً بشـــكل مباشر، إلا أن ذلك نادراً ما يتم في الممارسة الهندسية؛ حيث تُستخدم مكتبات متخصصة توفر واجهات برمجية لتبسيط عملية التطوير، مما يتيح للمهندس التركيز على منطق التطبيق (**Application Logic**) دون الخوض في التفاصـــيل الدقيقة لبنية حزم البيانات.

4.1 تعريف الـ API وأهميته في بيئة إنترنت الأشياء (IoT):

بنـاءً على التعريف التقني المعتمـد من شركـة **IBM**، فـإن الواجهـة البرمجيـة (**API - Application Programming Interface**) هي:

'مجموعة من القواعد والبروتوكولات التي تُمكّن التطبيقات البرمجية المختلفة من التواصـــل وتبادل البيانات والوظائف فيما بينها، دون الحاجة لمعرفة التفاصيل البرمجية الداخلية لكل نظام.'

تتجلى أهمية الـــــ **API** في إنترنت الأشـــياء في كونه يعمل كطبقة تجريد تســـمح لأجزاء النظام المتباينة — من متصـفحات، تطبيقات سـحابية، ولوحات تحكم — بالتكامل بسـلاسـة، دون الحاجة لفهم آلية العمل الداخلية أو البنية الهيكلية لكل جزء (مثل اختلاف لغات البرمجة بين المتصـــفح والمتحكم). علاوة على ذلك، تضـــيف هذه الواجهات طبقة أمان حيوية للنظام، حيث تعمل كبوابة تنظم صـــلاحيات الوصـــول وتخفي تفاصـــيل العتاد والبرمجيات الحساسة عن الأطراف الخارجية.



4.2 الاحداث الرئيسية (Main Events)

تُعد هذه الأحداث هي المحرك الأســـاسي لواجهة البرمجة (API)، حيث تتيح للمطور التحكم الكامل في دورة حياة الاتصال:

1. فتح الاتصال (**onopen**)

يتم استدعاؤه تلقائياً عند نجاح إنشاء الاتصال بين الطرفين.

```
socket.onopen = function() {
    console.log("Connection opened");
};
```

2. استقبال رسالة(**onmessage**)

يتم استدعاؤه فور وصول بيانات (نصوص أو ملفات ثنائية) من الطرف الآخر.

```
socket.onmessage = function(event) {
    console.log("Data received: " + event.data);
};
```

1. حدوث خطأ(**onerror**)

يتم استدعاؤه عند حدوث أي مشكلة تقنية أو فشل في الاتصال بالشبكة.

```
socket.onerror = function(error) {
    console.log("Error occurred");
};
```

2. إغلاق الاتصال(**onclose**)

يتم استدعاؤه عند انتهاء الجلسة أو إغلاق القناة بشكل منظم أو مفاجئ.

```
socket.onclose = function() {
    console.log("Connection closed");
};
```

- ملاحظة:

يجب التمييز بين نوعين من الأحداث عند بناء أنظمة**IoT**

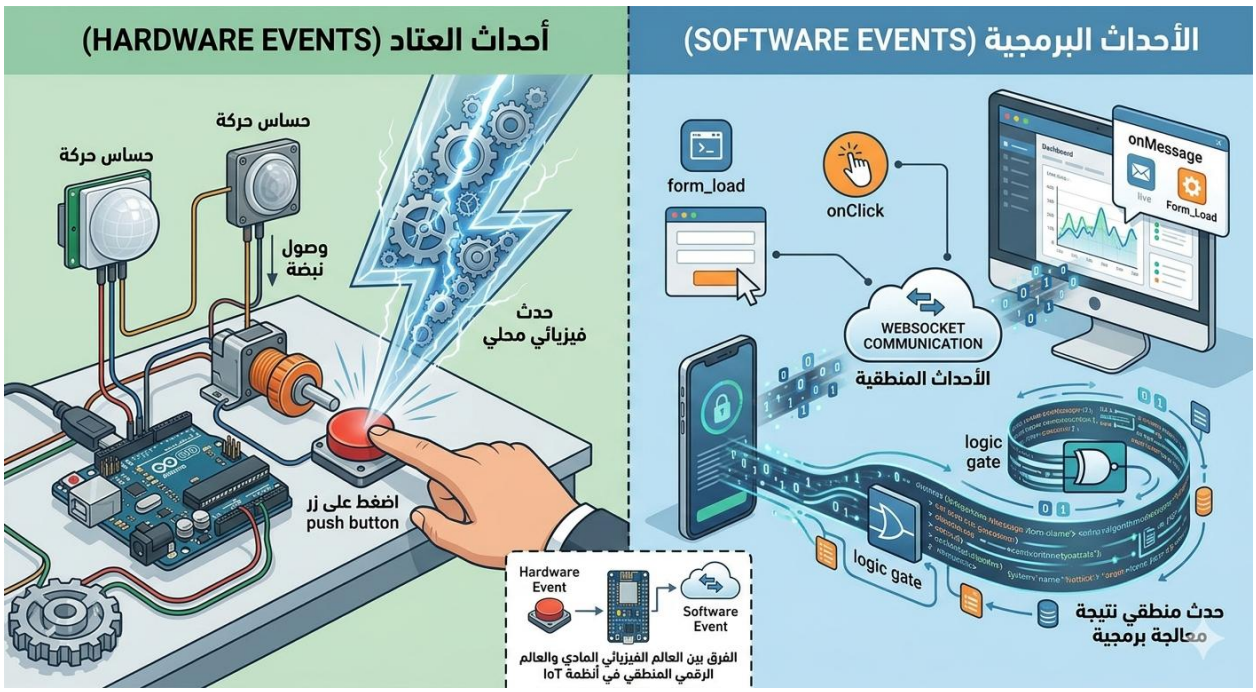
1. أحداث العتاد (**Hardware Events**): هي أحداث "فيزيائية" تتعلق باللوحة (**Controller**) ومكوناتها المادية.

أمثلة:

- ضغط الطالب على زر (**Push Button**)، أو وصول نبضة من حساس (**Interrupt**).
- الطبيعة: هي أحداث ملموسة تحدث "محلياً" داخل الدوائر الإلكترونية.

2. الأحداث البرمجية (**Software Events**): هي أحداث "منطقية" غير مادية، يتم إطلاقها بناءً على تغير في حالة النظام أو استجابة برمجية:

- في البرمجة التقليدية (**VB**): مثل حدث تحميل النموذج (**Form_Load**) أو النقر (**Click**).
- في الـــ **WebSocket**: هي أحداث تقع نتيجة معالجة برمجية؛ فلا يقع حدث (**onMessage**) إلا إذا استلم النظام بيانات، ولا يقع حدث (**onClose**) إلا عند تغير حالة الاتصال برمجياً.



5. اكواد الحالة (Error Codes)

تعتبر أكواد الحالة (Status Codes) في بروتوكول WebSocket (وبالأخص عند الإغلاق) جزءاً أساسيا من بنية البروتوكول فكما درسنا في بروتوكول HTTP تمكن هذه الاكواد المطور من تحديد حالة الاتصال وتفسير نوع العطل في حال حدوثه.

الجدول التالي يوضح أشهر اكواد الحالة Error Codes في WebSocket:

رقم الكود	المعنى	الشرح
1000	**Normal Closure**	تم إغلاق الاتصال بنجاح (مثلاً عند ضغط الطالب على زر "خروج" في الواجهة).
1001	**Going Away**	الطرف الآخر غادر مثل إغلاق المتصفح أو إعادة تشغيل لوحة(**ESP32**)
1002	**Protocol Error**	خطأ في قواعد البروتوكول (الطرفان لم يفهما لغة بعضهما).
1006	**Abnormal Closure**	الأكثر أهمية: انقطاع مفاجئ دون إرسال إشارة إغلاق (مثل فصل السلك أو فقدان الطاقة).
1011	**Internal Error**	وقوع خطأ برمجي غير متوقع داخل الخادم (**Server-side crash**)

6. التطبيقات التي يُستخدم فيها بروتوكول WebSockets

يستخدم بروتوكول WebSocket في التطبيقات التي تحتاج سرعة استجابة وتبادل بيانات عالية مثل:

1. أنظمة المراقبة والتحكم اللحظي (**Real-time Dashboards**)

تُستخدم بروتوكولات **WebSocket** في عرض بيانات الحساسات بشكل مباشر على واجهات الويب، مثل بيانات المسافة باستخدام حساس **Ultrasonic**، أو درجة الحرارة، أو جهد الأنظمة الشمسية.

الميزة: يتم نقل البيانات بزمن تأخير منخفض جداً (**Latency** أقل من 50**ms**)، مما يتيح للمســتخدم متابعة التغيرات فور حدوثها دون الحاجة إلى تحديث الصفحة.

2. التحكم في العتاد والروبوتات عن بُعد (**Remote Hardware Control**)

تُستخدم في تطبيقات التحكم المباشر بالأجهزة، مثل تحريك ذراع روبوتية أو قيادة سيارة ذكية عبر واجهة ويب.

الميزة: توفر اســتجابة فورية للأوامر؛ فعند الضـغط على زر مثل "**Forward**" يتم تنفيذ الأمر مباشرة. على عكس بروتوكول **HTTP**، الذي قد يُسبب تأخيراً ملحوظاً قد يؤثر سلباً على أداء النظام.

3. التطبيقات الصناعية (**Industrial Automation**)

تُستخدم لربط المتحكمات الصناعية (**PLCs**) بواجهات مراقبة عبر الويب، كما في أنظمة **SCADA** الحديثة.

الميزة: تمكّن المشــغّل من متابعة حالة الأنظمة (مثل المحركات والصــمامات) بشــكل لحظي داخل غرف التحكم، مما يعزز سرعة اتخاذ القرار.

4. تطبيقات الدردشة والتواصل التفاعلي (**Chat Applications**)

تُستخدم في تطبيقات المراسلة الفورية عبر الويب، مثل **WhatsApp Web** أو أنظمة الدعم الفني.

الميزة: تتيح إرســال واســتقبال الرســائل فورياً دون إعادة تحميل الصــفحة، بالإضــافة إلى ميزات تفاعلية مثل ظهور حالة "**User is typing**..." بشكل لحظي.

5. الألعاب متعددة اللاعبين عبر الإنترنت (**Multiplayer Games**)

تُستخدم في الألعاب التي تتطلب تفاعل عدة لاعبين في الوقت الحقيقي عبر المتصفح.

الميزة: تضـــمن مزامنة دقيقة لحركات اللاعبين ومواقعهم بزمن تأخير منخفض، مما يوفر تجربة لعب ســـلســـة ومتزامنة.

7. عيوب بروتوكولWebSocket

عيوب بروتوكول WebSocket

على الرغم من المزايا الكبيرة التي يقدمها بروتوكول WebSocket في تطبيقات الزمن الحقيقي، إلا أن هناك بعض القيود التي يجب أن تُؤخذ في الحسبان من حيث التعقيدات البرمجية واستهلاك موارد النظام، ومن أبرزها:

1. استهلاك الموارد (**Resource Consumption**)

يحافظ **WebSocket** على الاتصال مفتوحاً بشكل دائم (**Persistent Connection**)، مما يستهلك موارد الخادم (**Server**) مثل الذاكرة (**Memory**) التي تُستهلك بشكل متزايد كلما زاد عدد الاتصالات المتزامنة.

2. تعقيد التنفيذ (**Implementation Complexity**)

لاعتماد **WebSocket** برمجياً على التعامل مع "الأحداث" (**Events**) مثل تلقي رسالة أو انقطاع الاتصال، فهو يُعد أكثر تعقيداً مقارنة بـ **HTTP**؛ حيث يتوجب على المطور إدارة حالات الاتصال (**Connection State**)، ومعالجة الانقطاع، وإعادة الاتصال (**Reconnection**) بشكل يدوي ومستمر.

3. عدم التوافق الكامل مع بعض الشبكات (**Compatibility Issues**)

بعض الجدران النارية (**Firewalls**) أو الخوادم الوسيطة (**Proxies**) القديمة قد لا تدعم **WebSocket** بشكل كامل، مما قد يؤدي إلى حظر الاتصال أو فشله في بعض البيئات الشبكية المعقدة.

4. صعوبة التوسع (**Scalability Challenges**)

تظهر الحاجة إلى بنية تحتية متقدمة عند العمل على تطبيقات تتطلب الاتصال بعدد كبير جداً من المستخدمين، حيث يصعب توزيع الأحمال (**Load Balancing**) على خوادم متعددة مع الحفاظ على استمرارية الجلسات المفتوحة.

5. الأمان (**Security Concerns**)

يتطلب تأمين **WebSocket** عناية خاصة، مثل الالتزام باستخدام البروتوكول المشفر (**WSS**)، والتحقق الصارم من الهوية (**Authentication**). في حال الإهمال، قد يكون النظام عرضة لهجمات مثل:

- اعتراض البيانات (**Man-in-the-Middle**): التلصص على البيانات المتبادلة.
- استغلال الجلسات (**Session Hijacking**): سرقة هوية المستخدم المتصل.

- الخلاصة

على الرغم من كفاءة **WebSocket** في التطبيقات ذات الزمن الحقيقي، إلا أن اختياره كبروتوكول اتصال يجب أن يأتي بعد دراسة دقيقة لحاجة النظام الفعلية للاتصال السريع والمستمر، وموازنة ذلك مع إمكانيات النظام المتاحة من حيث توفر الموارد (المعالج والذاكرة) وضمان استقراريه الاتصال.

الجزء العملي

1. تمهيد الجزء العملي

في هذا الجزء، سيتم تنفيذ مجموعة من المشاريع العملية لربط نظام مدمج (Embedded System) مبني على لوحة WEMOS مع واجهة مستخدم (User Interface)، بهدف عرض قراءات الحساسات (Sensor Readings) على المتصفح، بالإضافة إلى إرسال أوامر التحكم (Control Commands) إلى اللوحة باستخدام بروتوكول الاتصال WebSocket.

ولغرض المقارنة بين أداء بروتوكولي الاتصال HTTP و WebSocket من حيث التأخير الزمني (Latency) والتذبذب (Jitter)، سيتم إعادة تنفيذ نفس المشاريع التي تم تطبيقها باستخدام بروتوكول HTTP، ولكن هذه المرة باستخدام WebSocket، وذلك لإرسال قراءات الحساسات من لوحة WEMOS إلى صفحة الويب بشكل لحظي.

سيتم إعادة تنفيذ التجربة 1 الجزء الثالث التي نفذت في جزء العملي الخاص ببروتوكول HTTP بحيث يستبدل البروتوكول ب WEBSOCKET وعبر فتح نافذ المطور على المتصفح سيتم اخذ القراءات ومقارنتها مع ما تم توثيقه سابقا.

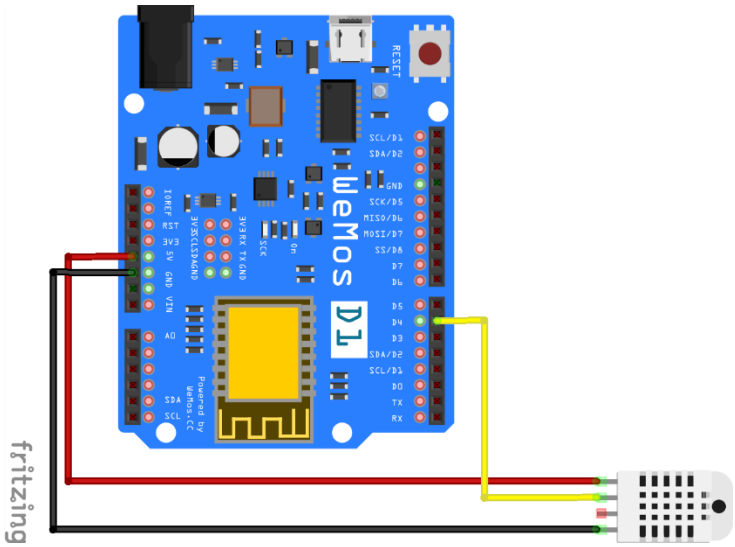
يتضمن هذا الجزء تنفيذ مشروع متكامل لمراقبة مستوى المياه (Water Level Monitoring) والتحكم في عملية تعبئة الخزانات. تعتمد آلية العمل على إرسال بيانات مستوى الماء من خزانين (Water Levels)إلى لوحة Wemos D1، والتي تقوم برمجياً بمقارنة هذه المستويات بالقيم المطلوبة .(Setpoints) وبناءً على هذه المقارنة، تُصدر اللوحة إشارات تحكم لتشغيل أو إيقاف مضخة المياه (Water Pump) آلياً إضافة الى صمام تفريغ (Drain Valve).

ويجسد هذا المشروع مفهوم "المكونات المادية داخل الحلقة - (Hardware In The Loop " (HIL، من خلال دمج المكونات المادية (Sensors & Actuators) مع بيئة برمجية تفاعلية (Interactive Software Environment)، مما يتيح تحقيق تحكم دقيق واستجابة فورية من خلال المراقبة اللحظية.(Real-time Monitoring)

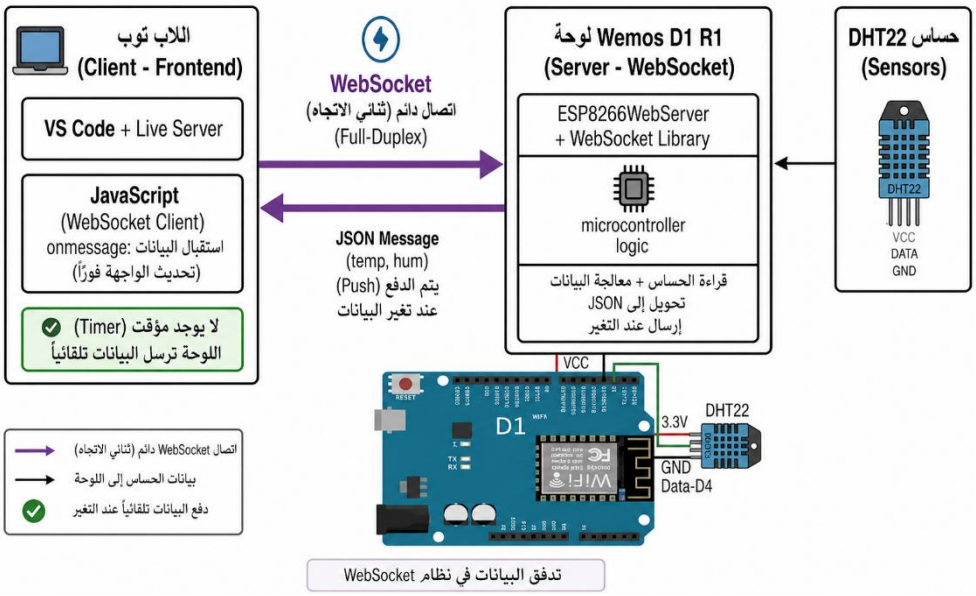
1.1 مكونات الدائرة:

1. لوحة **Wemos d1 r1** .
2. حساس الرطوبة والحرارة **DHT22**.

1.2 توصيل الدائرة:



1.3 المخطط الصندوقي



1.4 التجربة 1:

في هذه التجربة سيتم عرض قراءات الحرارة والرطوبة على واجهة المستخدم عبر بروتوكول الاتصال WebSocket. الشكل في الأسفل يوضح شكل واجهة المستخدم.

1.4.1 التعديلات على كود **Wemos**

1. استبدال مكتبة **HTTP** بمكتبة **WebSocket**

```
#include <WebSocketsServer.h>
```

2. تحديد منفذ الاتصال 81 ليستخدمه **Websocket**

```
WebSocketsServer webSocket = WebSocketsServer(81);
```

3. داخل دالة التهيئة **setup**: تشغيل خادم **Websocket** وتفعيل دالة التعامل مع الاحداث

```
webSocket.begin();
webSocket.onEvent(webSocketEvent);
Serial.println("WebSocket Server Started on Port 81");
```

4. إضافة دالة التعامل مع الاحداث الخاصة ب **Websocket**

```
void webSocketEvent(uint8_t num, WStype_t type, uint8_t * payload,
size_t length) {
  switch(type) {
    case WStype_DISCONNECTED:
      Serial.printf("[%u] Disconnected!\n", num); // حدث انقطاع الاتصال
      break;
    case WStype_CONNECTED:
    // حدث اتصال جديد
      Serial.printf("[%u] Connected from url: %s\n", num, payload);
      break;
    case WStype_TEXT:
      استقبال أوامر من المتصفح
      Serial.printf("[%u] Received Text: %s\n", num, payload);//
      break;
  }
}
```

5. تعديل دالة **read|_and_brodcast** لترسل القراءات عبر بروتوكول **Websocket**

```
webSocket.broadcastTXT(buffer);
```

1.4.2 كود **js**

1. رابط **WebSocket**

```
const gateway = `ws://192.168.1.13:81/`;
```

2. انشاء الاتصال:

```
 socket = new WebSocket(gateway);
```

3. حدث انقطاع الاتصال :محاولة إعادة الاتصال في حال الانقطاع مرة كل 3 ثواني

```
socket.onclose = () => setTimeout(initWebSocket, 3000);
```

4. حدث استقبال رسالة البيانات:

كما في تجربة **HTTP** توجيه البيانات الى ما يقابلها في الواجهة.

```
socket.onmessage = (event) => {
          const data = JSON.parse(event.data);
          console.log("بيانات مستلمة: ", data);
          // تحديث النصوص في الواجهة
          document.getElementById("temp-value").textContent = data.temp + "°C";
          document.getElementById("humi-value").textContent = data.hum + "%";

          // تحديث الدوائر الرسومية عبر متغيرات CSS
          const tempCircle = document.getElementById("temp-circle");
          const humiCircle = document.getElementById("humi-circle");

          tempCircle.style.setProperty("--p", data.temp);
          tempCircle.style.setProperty("--max", 50); // أقصى درجة حرارة متوقعة

          humiCircle.style.setProperty("--p", data.hum);
          humiCircle.style.setProperty("--max", 100); // 100 أقصى رطوبة%
        };
```

1.4.3 مقارنة الأداء

من الشكل في الأسفل الذي يعرض أداء الشبكة على المتصفح

Name	Status	Type	Initiator	Size	Time	Ful...
lab_ambient_websocket.html	304	document	Other	243 B	3 ms	
ws	101	websock...	lab_ambient_websock	0 B	Pending	
192.168.1.13	101	websock...	lab_ambient_websock	0 B	Pending	

وخلال تحليل أداء الشبكة الموضح في الشكل، يتضح التفوق الواضح لبروتوكول WebSocket في تقليل حجم البيانات الإضافية (Overhead) المرتبطة بإنشاء الاتصالات المتكررة.

فعلى عكس بروتوكول HTTP الذي يتطلب إرسال طلب جديد (Request) وإنشاء دورة اتصال كاملة عند كل عملية جلب بيانات، يحافظ المتصفح في WebSocket باتصال دائم (Persistent Connection)، كما يظهر من الحالة المستمرة (Pending) في أدوات المطور.

هذا الاتصال المستمر يسمح بنقل البيانات بشكل مباشر وفوري (Real-time) دون الحاجة إلى إعادة تهيئة الاتصال في كل مرة، مما يؤدي إلى:

- تقليل عدد الحزم المتبادلة عبر الشبكة
- خفض زمن الاستجابة (**Latency**)
- تقليل الحمل على المعالج في كل من المتصفح ولوحة التحكم (**ESP8266**)
- منع ازدحام الشبكة الناتج عن الطلبات المتكررة (**Polling**)

وبالتالي، يوفر هذا الأسلوب كفاءة أعلى وموثوقية أفضل في تطبيقات المراقبة اللحظية (Real-time Monitoring).

الشكل التالي يوضح تحليل الشبكة لنفس الواجهه و اللوحة لكن باستخدام **http**

Name	Status	Type	Initiator	Size	Time	Fulfill...
	OK	fetch	Script	22 B	16 ms	
192.168.1.13	200 OK	fetch	lab_ambient_monitoring.html:2 Script	175 B 22 B	82 ms 22 ms	
192.168.1.13	200 OK	fetch	lab_ambient_monitoring.html:2 Script	175 B 22 B	69 ms 19 ms	
192.168.1.13	200 OK	fetch	lab_ambient_monitoring.html:2 Script	175 B 22 B	81 ms 24 ms	
192.168.1.13	200	fetch	lab_ambient_monitoring.html:2	175 B	68 ms	

يمكننا الان تكوين جدول مقارنة للأداء بين الشكل 1 والشكل 2 كما يلي

وجه المقارنة	WebSocket	HTTP	ملاحظات تقنية
حجم الترويسة (Header Size)	22 Byte	175 Byte	**تخلصنا من تكرار بيانات المتصفح (User-Agent, Cookies) في كل رسالة.**
زمنالتأخير (Latency)	10ms	1000ms	**في HTTP المحرك هو" مؤقت(Timer) "، وفي WebSocketالمحرك هو" الحدث.(Event) "**
كفاءة النقل (Efficiency)	90%فأكثر	12.5%تقريباً	**أغلب حجم حزمة HTTP مكون من الترويسات، بينما أغلب حزمة WebSocket بيانات فعلية.**
حالة الاتصال (Status)	101 Switching Protocols	200 OK	**يظهر في" شاشة المطور "تحول البروتوكول عند المصافحة الأولى.(Handshake)**
عدد الطلبات (Requests)	اتصال واحد مستمر	طلب جديد لكل قراءة	**يقلل الجهد على معالج لوحة.Wemos D1 R1**

- ملاحظة

لا يقتصر الفرق بين البروتوكولين على سرعة نقل البيانات فحسب، بل يمتد ليشمل استدامة العتاد المادي .(Hardware) فاستخدام بروتوكول HTTP مع المعالجات أحادية النواة) مثل ESP8266)يجبر المعالج على البقاء في حالة معالجة قصوى لتفكيك ترويسات الطلبات المتكررة، مما يؤدي لارتفاع ملحوظ في درجة حرارته. في المقابل، وبفضل خفة إطارات (Frames)بروتوكولWebSocket ، يعمل المعالج بكفاءة أعلى وحمل أقل، مما يضمن استقرار النظام وحمايته من التلف الحراري على المدى الطويل.

2. التجربة 2

سـيتم تنفيذ مشـروع متكامل لمراقبة مسـتوى المياه (Water Level Monitoring) والتحكم في عملية تعبئة الخزانات والتفريغ. تعتمد آلية العمل على إرسـال بيانات مسـتوى الماء من خزانين (Water Levels)إلى لوحة Wemos D1، والتي تقوم برمجياً بمقارنة هذه المسـتويات بالقيم المطلوبة .(Setpoints) وبناءً على هذه المقارنة، تُصـدر اللوحة إشـارات تحكم لتشـغيل أو إيقاف مضخة المياه (Water Pump) آلياً إضافة الى صمام التفريغ (Drain Valve).

2.1 مكونات النظام

2.1.1 الواجهة

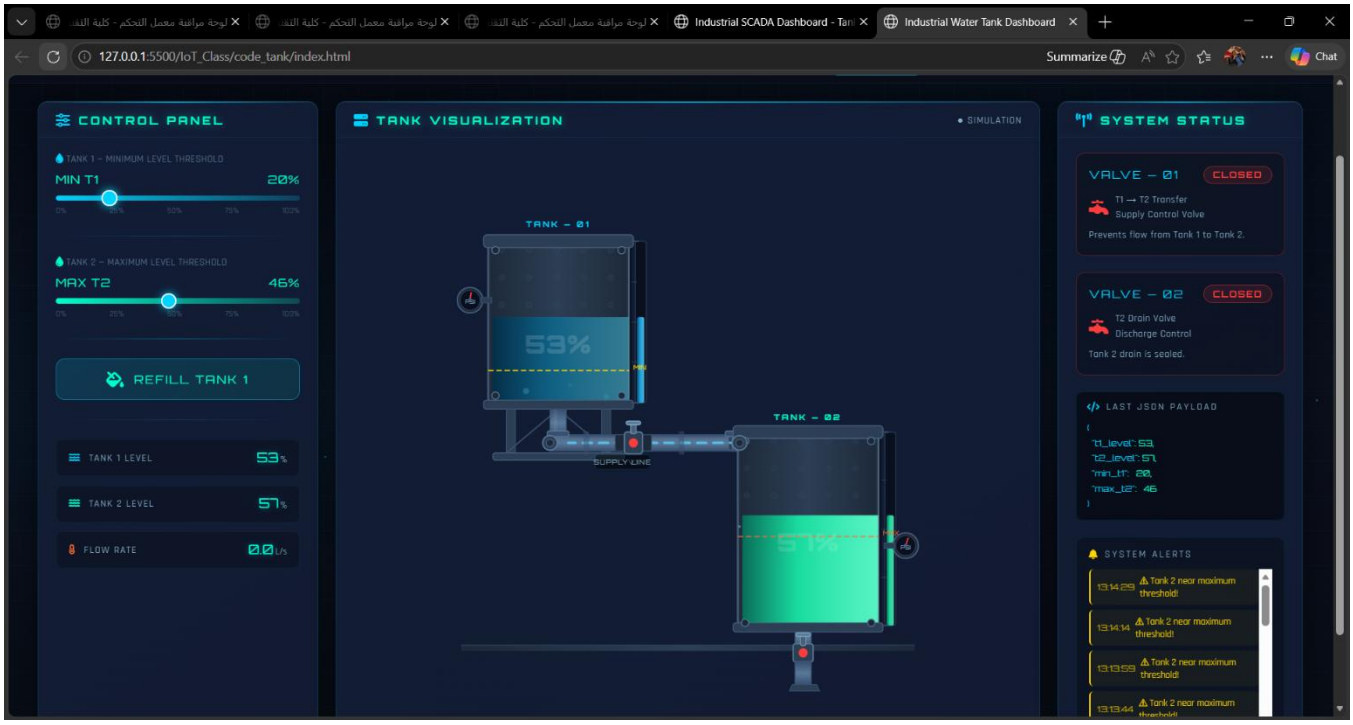
تتكون واجهة المستخدم من:

تتكون واجهة المستخدم من العناصر التفاعلية التالية:

1. الخزان الرئيسي- **(Tank-01)**: يمثل مصـدر المياه الأسـاسي في النظام. يمكن للمسـتخدم إعادة تعبئته يدوياً عند الحاجة عبر الضغط على زر **Refill Tank1** الموجود في يسار الواجهة.
2. الخزان الفرعي **(Tank-02)**: هو الخزان المسـتهدف بالتحكم ومراقبة مسـتوى الماء. يحتوي في قاعدته على صمام تفريغ **(Drain Valve)** يتم التحكم في فتحه وإغلاقه برمجياً عبر لوحة **Wemos D1**.
3. محددات مستوى الماء **(Level Selectors)**: توجد منزلقات التحكم **(Sliders)** في الجهة اليسرى من الواجهة لضبط معايير الأمان والتحكم:
   - المحدد الأول **(Min Level - Tank1)**: يحدد الحد الأدنى المسموح به للماء في الخزان الأول؛ حيث تُعطى إشـارة لإيقاف المضـخة فوراً عند وصـول الماء لهذا المسـتوى لحماية النظام من التشغيل الجاف.
   - المحدد الثاني **(Max Level - Tank2)**: يحدد الحد الأعلى المطلوب للماء في الخزان الثاني؛ وبناءً عليه تتوقف المضخة آلياً لمنع تجاوز المستوى المطلوب أو فيضان الخزان.
4. إعدادات الاتصال والربط: توفر الواجهة خانة مخصصة لإدخال عنوان الـ **IP** الخاص باللوحة، بالإضافة إلى زر اتصال **(Connect)**. تضمن هذه الخاصية مرونة عالية في الاسـتخدام، حيث تتيح ربط الواجهة بأي لوحة عبر الشبكة دون الحاجة لتعديل العنوان برمجياً داخل الكود **(Hardcoding)**.

تعرض الواجهة مجموعة من الإحصائيات الحيوية الضرورية لمتابعة أداء النظام بدقة؛ حيث تضمن تزويد المستخدم بكافة المؤشرات اللحظية، مثل القيم العددية الدقيقة لمستويات المياه وحالات التشغيل (On/Off) للمضخة والصمام. تهدف هذه البيانات إلى تمكين المستخدم من تكوين رؤية شاملة ودقيقة حول حالة النظام لحظة بلحظة، مما يسهل عملية الرصد الاستباقي لأي خلل أو تجاوز للمستويات المسموح بها، وذلك بفضل سرعة تحديث البيانات التي يوفرها بروتوكول **WebSocket** المستخدم في الربط بين اللوحة والواجهة.

الشكل بالأسفل يوضح شكل واجهة المستخدم ومحتوياتها



2.1.2 كود **java script**

1. تحديث القيم عند تحريك **Slider** حيث يتم تحويل القيمة القادمة منه (وهي اقل مستوى للماء مسموح به للخزان 1) من نص الى رقم حتى يتم تحديث القراءة على الواجهة وارســال القيم الى اللوحة للتحكم في المضخة

```
// ── Slider callbacks ──────────────────────────────────────────
      function onSlider1Change(val) {
        state.minTank1 = parseInt(val);
        document.getElementById("slider1-val").textContent = val + "%";
        setMinLineT1(state.minTank1);
        renderJSONDisplay();
        scheduleSend();
      }
```

2. الدالة المسئولة عن وضع البيانات في صيغة **json**

```
function buildSendPayload() {
        return {
          t1_level: Math.round(state.tank1),
          t2_level: Math.round(state.tank2),
          min_t1: state.minTank1,
          max_t2: state.maxTank2,
        };
      }
```

3. الدالة المسئولة عن ارسال البيانات بصيغة **json** الى اللوحة بمعدل ارسال كل نصف ثانية

```
function sendData() {
        if (!state.ws || state.ws.readyState !== WebSocket.OPEN) return;
        const payload = JSON.stringify(buildSendPayload());
        state.ws.send(payload);
        state.txCount++;
        state.totalMsg++;
        document.getElementById("ws-tx-count").textContent = state.txCount;
        document.getElementById("footer-ws-msg").textContent = state.totalMsg;
      }

            setInterval(() => {
        if (state.wsConnected) sendData();
      }, 500);
```

4. دالة **function toggleWS()**

هذه الدالة هي المسئولة عن بدء الاتصال و مراقبته و التعامل مع احداث انقطاع الاتصال

- هذا الجزء مسئول عن قراءة **ip** من الخانة المخصصة بالواجهة و التحقق من مطابقته للصيغة المعتمدة.

```
const url = document.getElementById("ws-ip-input").value.trim();
        if (!url) {
          addAlert("Enter a valid WebSocket URL.", "error");
          return;
        }
```

- مراقبة الاتصال وطباعة الخطأ في حال حدوثه

```
try {
          state.ws = new WebSocket(url);
        } catch (e) {
          setWSStatus("disconnected");
          addAlert("Invalid WebSocket URL: " + e.message, "error");
          return;
        }
```

- عند نجاح الاتصال تطبع على الواجهة ان الاتصال ناجح و العنوان المتصل به وطالما الاتصال ناجح تستدعي دالة ارسال البيانات **sendData()**

```
 state.ws.onopen = () => {
          setWSStatus("connected");
          setWSLog("Connected to " + url);
          addAlert("WebSocket connected! Live data active.", "ok");
          document.getElementById("ws-target-display").textContent = url;
          sendData();
        };
```

- التعامل مع الرسائل والواردة من اللوحة واستدعاء دالة **handleWemosData()** للتعامل مع البيانات وتوجيهها.

```
state.ws.onmessage = (evt) => {
          state.rxCount++;
          state.totalMsg++;
          document.getElementById("ws-rx-count").textContent = state.rxCount;
          document.getElementById("footer-ws-msg").textContent = state.totalMsg;
          try {
            const data = JSON.parse(evt.data);
            handleWemosData(data);
            setWSLog("RX: " + evt.data.substring(0, 80));
          } catch (e) {
            addAlert(
              "Invalid JSON from Wemos: " + evt.data.substring(0, 40),
              "warn",
            );
          }
        };
```

5. الدالة المسئولة عن توجيه أوامر التحكم من اللوحة الى العناصر الصحيحة على الواجهة
   - هذه الدالة تقبل **(1, true , open)** لتشغيل المضخة او فتح صمام الخزان 2.
   - أي بيانات غيرها تعتبر امر إطفاء المضخة او اغلاق الصمام

```
function handleWemosData(data) {
        let changed = false;

        if (data.valve1 !== undefined) {
          const open =
            data.valve1 === "open" || data.valve1 === true || data.valve1 === 1;
          if (open !== state.valve1Open) {
            state.valve1Open = open;
            setValve1Visual(open);
            addAlert(
              `Wemos → Valve 1 ${open ? "OPENED" : "CLOSED"}.`,
              open ? "ok" : "info",
            );
            changed = true;
          }
        }
```

```
      if (data.valve2 !== undefined) {
        const open =
          data.valve2 === "open" || data.valve2 === true || data.valve2 === 1;
        if (open !== state.valve2Open) {
          state.valve2Open = open;
          setValve2Visual(open);
          addAlert(
            `Wemos → Valve 2 ${open ? "OPENED" : "CLOSED"}.`,
            open ? "ok" : "info",
          );
          changed = true;
        }
      }
```

2.1.3 كود **Wemos**

1. استدعاء كل من:

- دالة() **Pin_chack** للتحقق من حالة **pin** (الضغط على زر تشغيل او اطفاء المضخة، الضغط على زر فتح وقفل صمام تفريغ الخزان الثاني).
- دالة **read_and_broadcast()** لارسال البيانات الى الواجهة

وذلك كل نصف ثانية.

```
if (millis() - lastUpdate > interval) {
  lastUpdate = millis();
  Pin_chack();
  read_and_broadcast();
}
```

2. دالة **webSocketEvent()**

هذه الدالة مسـئولة عن التعامل مع الاحداث الخاصـة ب **WebSocket**, فعند اسـتقبال سـالة من الواجهة تحتوي على (مستوى الماء الحالي في الخزانين, مستوى الماء المطلوب) تقوم بتفكيكه من صورة **json** .

```
case WStype_TEXT:
      // استقبال البيانات وتفكيك JSON القادم من المتصفح
      StaticJsonDocument<256> doc;
      DeserializationError error = deserializeJson(doc, payload);


      if (!error) {
        if (doc.containsKey("t1_level")) {
          currentL1 = doc["t1_level"];
          currentL2 = doc["t2_level"];
          limitMin1 = doc["min_t1"];
          limitMax2 = doc["max_t2"];
        //   Change_Valve_State();
          // يمكنك هنا إضافة Logic للتحكم بريلي (Relay) بناءً على القيم
          Serial.printf("Received Tank Data -> L1: %.1f, L2: %.1f,
Min1: %d, Max2: %d\n",
                        currentL1, currentL2, limitMin1, limitMax2);
        }
      }
```

3. دالة **Change_Valve_State()**

هذه الدالة هي المسئولة عن تغيير حالة الصمامات حسب البيانات الواردة من الواجهة

```
void Change_Valve_State(){
      if(currentL1<=limitMin1 || currentL2>=limitMax2){
              Valve1_State=false;
      }
      if(currentL2<=limitMax2){
              Valve2_State=false;
      }
}
```

4. دالة **read_and_broadcast()**

هذه الدالة مسئولة عن قراءة البيانات (حالة الصمامات) وتحويلها الى صورة **josn** ارسالها الى الواجهة.

```
void read_and_broadcast() {
  StaticJsonDocument<128> doc;
  Change_Valve_State();
  doc["valve1"] = Valve1_State;
  doc["valve2"] = Valve2_State;
  char buffer[128];
  serializeJson(doc, buffer);
  webSocket.broadcastTXT(buffer);
  Serial.print("Sent to Clients: ");
  Serial.println(buffer);
}
```

المراجع (References)

- أكاديمية حسوب (2023). موسوعة حسوب البرمجية: بروتوكول **WebSocket**. [مرجع إلكتروني].
- مجمع اللغة العربية (2020). معجم مصطلحات تقنية المعلومات. القاهرة، جمهورية مصر العربية.
- **Fette, I., & Melnikov, A. (2011). RFC 6455: The WebSocket Protocol. Internet Engineering Task Force (IETF)**.
- **Grigorik, I. (2013). High Performance Browser Networking: What every web developer should know about networking and browser performance. O'Reilly Media, Inc**.
- **Kurniawan, A. (2018). IoT Projects with Arduino Nano 33 BLE Sense: For Healthcare Applications. Apress**.
- **Lombardi, V. (2015). The Definitive Guide to HTML5 WebSocket. Apress**.
- **Maier, A., Sharp, A., & Vagrunov, Y. (2017). Hands-On Industrial Internet of Things. Packt Publishing**.
- **Santos, R., & Santos, S. (2021). SMART HOME with Raspberry Pi and ESP32. Random Nerd Tutorials**.